

UNISINOS

RELATÓRIO TÉCNICO DETALHADO

Laboratório de Roteamento em Ubuntu Linux, Docker e FRRouting (FRR)

Implementação de cinco roteadores virtuais, três Sistemas Autônomos e análise de RIP, OSPF e BGP

Aluno: Eduardo Cansan

Setembro de 2026

1. Visão geral do trabalho

Este documento registra de forma detalhada o processo realizado no laboratório em Ubuntu Linux. O objetivo foi transformar uma única máquina virtual em um ambiente de roteamento composto por cinco roteadores lógicos independentes. Para isso, o Docker foi utilizado para isolar os roteadores em containers e o FRRouting (FRR) forneceu os protocolos e as tabelas de roteamento.

A documentação abaixo prioriza o que foi efetivamente construído, configurado, observado e corrigido durante o experimento. Etapas meramente tentadas e abandonadas não são tratadas como resultado final. Quando uma métrica não foi coletada numericamente no Ubuntu, isso é indicado explicitamente em vez de preencher o relatório com valores estimados.

O laboratório foi organizado em três Sistemas Autônomos: AS100, AS200 e AS300. O R1 representa a extremidade do AS100; R2, R3 e R4 formam o núcleo do AS200; e R5 representa a extremidade do AS300. A estrutura permite observar tanto roteamento interno quanto troca de rotas entre AS diferentes.

Elemento	Implementação utilizada
Sistema operacional	Ubuntu Linux executado em máquina virtual
Virtualização da rede	Docker
Roteadores	Containers r1, r2, r3, r4 e r5
Suíte de roteamento	FRRouting (FRR)
CLI de roteamento	vttysh
Protocolos estudados	RIP, OSPF e BGP
Validação	Tabelas de rotas, vizinhanças/sessões, ping e traceroute

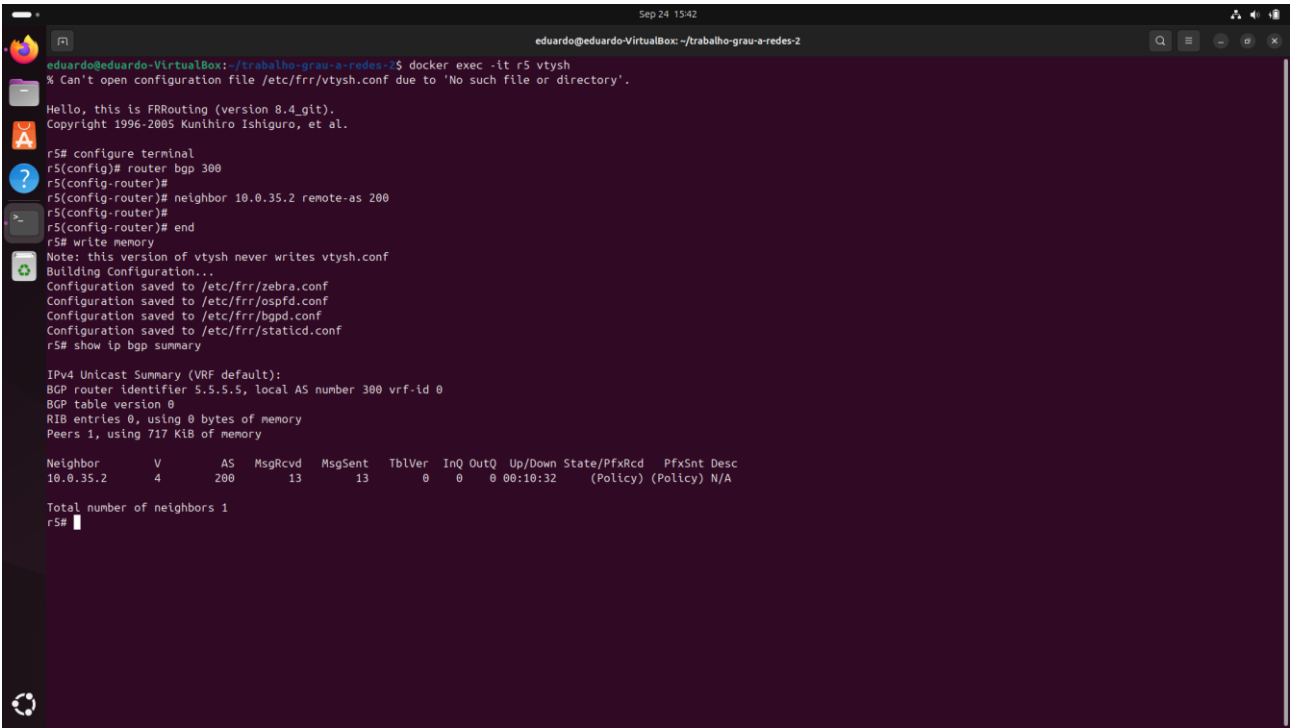


Figura 1 - Terminal do Ubuntu utilizado na execução e administração do laboratório.

2. Arquitetura física: VM Ubuntu + Docker

Fisicamente, todo o experimento foi executado em uma única VM Ubuntu. Não havia cinco computadores físicos: a separação dos roteadores foi obtida por virtualização em nível de sistema operacional. O Docker criou namespaces de rede e interfaces virtuais para que cada container tivesse sua própria pilha de rede, interfaces, endereços e tabela de roteamento.

Essa abordagem é importante porque permite reproduzir, em um único notebook, o comportamento de uma rede com vários roteadores. O host Ubuntu fornece CPU, memória e kernel; o Docker fornece isolamento; e o FRR executado dentro de cada container assume a função de plano de controle de roteamento.

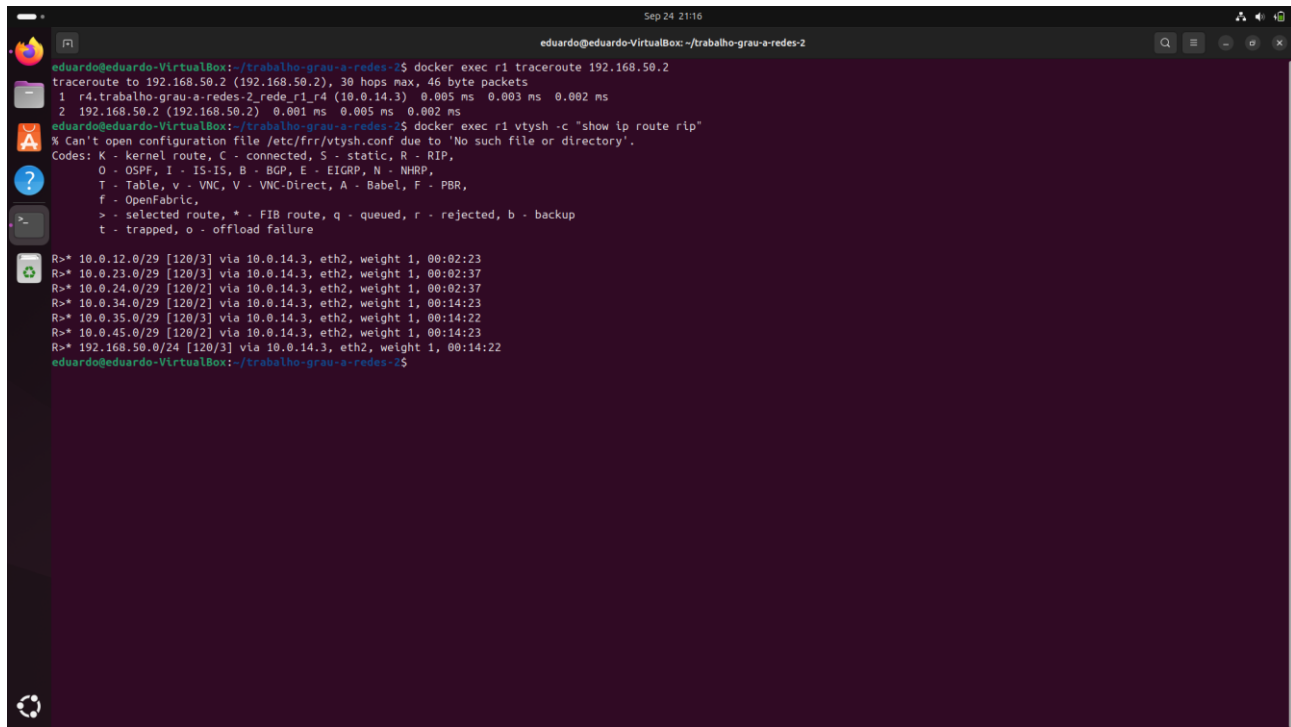
The image shows a terminal window titled 'eduardo@eduardo-VirtualBox: ~/trabalho-grau-a-redes-2'. The user is running several Docker commands. First, they run 'docker exec r1 traceroute 192.168.50.2', which shows a successful traceroute from r1 to r4. Then, they run 'docker exec r1 vtysh -c "show ip route rip"', which displays the RIP routing table for container r1, listing several routes with their metrics and interfaces. The terminal output includes details about the traceroute (30 hops max, 46 byte packets) and the contents of the routing table, such as 'R> 10.0.12.0/29 [120/3] via 10.0.14.3, eth2, weight 1, 00:02:23'.

Figura 2 - Ambiente Ubuntu mostrando os containers e o estado do laboratório.

A arquitetura pode ser entendida em três camadas: (1) Ubuntu como plataforma hospedeira; (2) Docker como camada de isolamento e interconexão; (3) FRRouting dentro de cada container como software de roteamento. Essa separação facilitou a inspeção individual de cada roteador por meio de comandos como `docker exec`.

3. Criação dos cinco roteadores lógicos

Os roteadores foram representados pelos containers r1, r2, r3, r4 e r5. Cada nome passou a representar um nó da topologia. A verificação do ambiente era feita no Ubuntu, confirmando que os cinco containers estavam em execução antes de iniciar os testes de roteamento.

```
docker ps
docker exec r1 vtysh
docker exec r2 vtysh
docker exec r3 vtysh
docker exec r4 vtysh
docker exec r5 vtysh
```

O comando `docker exec` permitiu entrar logicamente em cada roteador sem abrir outra máquina virtual. O `vtysh` agregou as interfaces de configuração do FRR e forneceu comandos de inspeção semelhantes aos encontrados em equipamentos de rede tradicionais.

```
eduardo@eduardo-VirtualBox: ~/trabalho-grau-a-redes-2$ docker exec r1 ip -br addr
lo                UNKNOWN      127.0.0.1/8 ::1/128
eth0@if17         UP                10.0.12.2/29
eth1@if23         UP                10.0.14.2/29
eth2@if26         UP                192.168.10.2/24
eduardo@eduardo-VirtualBox: ~/trabalho-grau-a-redes-2$ for r in r2 r3 r4 r5; do
echo "===== $r ====="
docker exec $r ip -br addr
done
===== r2 =====
lo                UNKNOWN      127.0.0.1/8 ::1/128
eth0@if18         UP                10.0.12.3/29
eth1@if25         UP                10.0.23.2/29
eth2@if30         UP                10.0.24.2/29
===== r3 =====
lo                UNKNOWN      127.0.0.1/8 ::1/128
eth0@if16         UP                10.0.23.3/29
eth1@if21         UP                10.0.34.2/29
eth2@if27         UP                10.0.35.2/29
===== r4 =====
lo                UNKNOWN      127.0.0.1/8 ::1/128
eth0@if14         UP                10.0.14.3/29
eth1@if19         UP                10.0.24.3/29
eth2@if24         UP                10.0.34.3/29
eth3@if28         UP                10.0.45.2/29
===== r5 =====
lo                UNKNOWN      127.0.0.1/8 ::1/128
eth0@if22         UP                10.0.45.3/29
eth1@if29         UP                192.168.50.2/24
eth2@if31         UP                10.0.35.3/29
eduardo@eduardo-VirtualBox: ~/trabalho-grau-a-redes-2$
```

Figura 3 - Verificação dos roteadores/containers e comandos executados a partir do Ubuntu.

```
eduardo@eduardo-VirtualBox: ~/trabalho-grau-a-redes-2$ docker exec -it r1 vtysh
% Can't open configuration file /etc/frr/vtysh.conf due to 'No such file or directory'.

Hello, this is FRRouting (version 8.4.git).
Copyright 1996-2005 Kunihiko Ishiguro, et al.

r1# configure terminal
r1(config)# router bgp 100
r1(config-router)# bgp router-id 1.1.1.1
r1(config-router)# network 192.168.10.0/24
r1(config-router)# end
r1# show running-config
Building configuration...

Current configuration:
!
frr version 8.4.git
frr defaults traditional
hostname r1
no ipv6 forwarding
!
interface eth2
 ip ospf cost 100
exit
!
router bgp 100
 bgp router-id 1.1.1.1
!
 address-family ipv4 unicast
  network 192.168.10.0/24
 exit-address-family
exit
!
router ospf
 network 10.0.12.0/29 area 0
 network 10.0.14.0/29 area 0
 network 192.168.10.0/24 area 0
exit
!
end
r1#
```

Figura 4 - Inspeção do ambiente de rede dos containers no Linux.

4. Topologia lógica e endereçamento

A topologia lógica foi desenhada para possuir múltiplos caminhos entre as extremidades. O R1 conecta-se ao núcleo AS200 por R2 e R4. Dentro do AS200 existem ligações entre R2, R3 e R4. O AS300 é alcançado pelo R5, conectado ao núcleo por R3 e R4. Essa redundância permite que os protocolos escolham caminhos e que mudanças de custo ou disponibilidade alterem a decisão de roteamento.

Ligação	Rede
R1 ↔ R2	10.0.12.0/29
R1 ↔ R4	10.0.14.0/29
R2 ↔ R3	10.0.23.0/29
R2 ↔ R4	10.0.24.0/29
R3 ↔ R4	10.0.34.0/29
R3 ↔ R5	10.0.35.0/29
R4 ↔ R5	10.0.45.0/29
LAN do AS100	192.168.10.0/24
LAN do AS300	192.168.50.0/24

As redes /29 foram usadas nos enlaces entre roteadores. As redes /24 representam as redes finais das extremidades. O destino utilizado na validação final do BGP foi 192.168.10.2, pertencente ao lado do AS100.

Topologia lógica do laboratório em Ubuntu + Docker + FRRouting

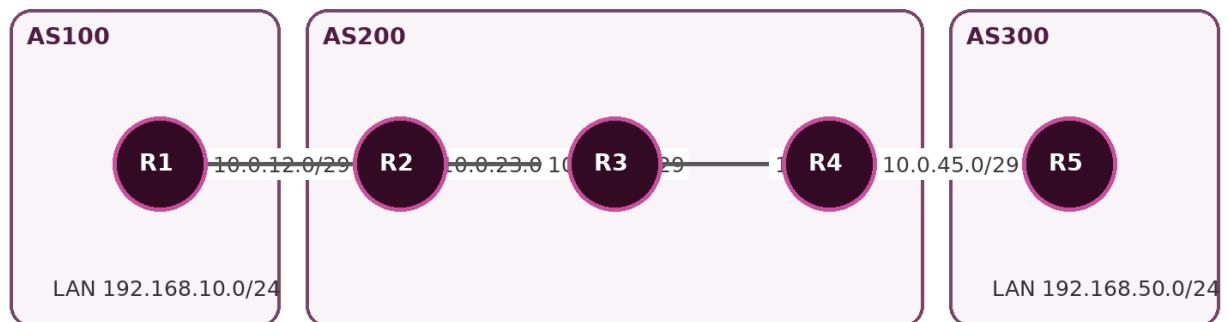


Figura 5 - Representação consolidada da topologia lógica Ubuntu/Docker/FRR.

5. Organização dos Sistemas Autônomos

Roteador	Sistema Autônomo	Função
R1	AS100	Roteador de borda e origem da rede 192.168.10.0/24
R2	AS200	Roteador interno/de borda do núcleo
R3	AS200	Roteador interno e ligação com AS300
R4	AS200	Roteador interno e caminho alternativo entre as extremidades
R5	AS300	Roteador de borda e origem da rede 192.168.50.0/24

A divisão em três AS foi essencial para o BGP. Vizinhos pertencentes a AS diferentes estabeleceram sessões eBGP; vizinhos que pertenciam ao AS200 utilizaram sessões internas para transportar informações de roteamento dentro do próprio sistema autônomo.

No experimento, o AS200 funciona como domínio de trânsito entre AS100 e AS300. Assim, uma rota originada no AS100 pode ser propagada pelo AS200 e finalmente recebida no AS300.

6. FRRouting: instalação lógica e administração

O FRRouting foi a suíte de roteamento usada dentro dos containers. O processo de administração foi feito pelo vtysh, que permite consultar e alterar a configuração dos daemons de roteamento. A partir dele foram observadas tabelas de rotas, vizinhos OSPF, sessões BGP, atributos BGP e rotas instaladas no kernel.

```
docker exec r5 vtysh -c "show ip bgp summary"
docker exec r5 vtysh -c "show ip bgp"
```

```
docker exec r5 vtysh -c "show ip route bgp"
docker exec r5 vtysh -c "show ip route 192.168.10.0/24"
```

Durante a execução apareceu repetidamente a mensagem “Can't open configuration file /etc/frr/vtysh.conf due to 'No such file or directory'”. Ela foi tratada como aviso do ambiente, pois os comandos do vtysh continuavam funcionando e as tabelas e sessões eram exibidas normalmente.



Figura 6 - Execução de comandos do FRRouting através do vtysh.

7. Teste com RIP no ambiente FRR

O RIP foi estudado como protocolo de vetor de distância. Seu critério de seleção é a quantidade de saltos até o destino. No FRR, as redes diretamente conectadas foram anunciadas e as rotas dinâmicas foram verificadas nas tabelas de cada roteador.

```
show ip protocols
show ip route
show ip route rip
ping <destino>
traceroute <destino>
```

A leitura da tabela permitiu distinguir redes diretamente conectadas das redes aprendidas dinamicamente. A interpretação do RIP foi feita relacionando a métrica apresentada ao número de roteadores atravessados. O teste de conectividade foi usado para confirmar que a existência de uma rota na tabela correspondia a um caminho utilizável.

```

Sep 24 12:06
eduardo@eduardo-VirtualBox: ~/trabalho-grau-a-redes-2

[+] Running 3/3
  Container r2                               Removed
  Container r1                               Removed
  Network trabalho-grau-a-redes-2_rede_r1_r2 Removed

eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ nano docker-compose.yml
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ nano docker-compose.yml
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ nano docker-compose.yml
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ nano docker-compose.yml
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ docker compose config
name: trabalho-grau-a-redes-2
services:
  r1:
    container_name: r1
    hostname: r1
    image: frrouting/frr:latest
    networks:
      rede_r1_r2:
        ipv4_address: 10.0.12.2
        privileged: true
  r2:
    container_name: r2
    hostname: r2
    image: frrouting/frr:latest
    networks:
      rede_r1_r2:
        ipv4_address: 10.0.12.3
        privileged: true
networks:
  rede_r1_r2:
    name: trabalho-grau-a-redes-2_rede_r1_r2
    driver: bridge
    ipam:
      config:
        - subnet: 10.0.12.0/29

eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ docker compose up -d
[+] Running 3/3
  Network trabalho-grau-a-redes-2_rede_r1_r2 Created
  Container r1                               Started
  Container r2                               Started

eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED     STATUS    PORTS    NAMES
3b024c1e4661   frrouting/frr:latest  "/sbin/tini -- /usr/"   7 seconds ago  Up 6 seconds    Up 6 seconds    r2
b630673fdf66   frrouting/frr:latest  "/sbin/tini -- /usr/"   7 seconds ago  Up 6 seconds    Up 6 seconds    r1

```

Figura 7 - Saídas de roteamento coletadas no Ubuntu/FRR durante os testes.

Não foi registrada, nos prints disponíveis, uma captura quantitativa padronizada de pacotes de controle, bytes de controle ou tempo de convergência do RIP. Portanto, esses valores não são inventados neste relatório.

8. Teste com OSPF no ambiente FRR

O OSPF foi utilizado para observar uma estratégia diferente do RIP. Em vez de escolher simplesmente o menor número de saltos, o OSPF calcula o menor custo acumulado. Antes de considerar uma rota OSPF válida, também foi necessário verificar a formação de adjacências entre os roteadores.

```
show ip protocols
show ip ospf neighbor
show ip ospf interface
show ip route ospf
show ip route
```

A presença de vizinhos em estado adequado indicou que os roteadores estavam trocando informações de estado de enlace. Em seguida, as rotas marcadas pelo OSPF foram comparadas com o caminho observado no traceroute.

```
eduardo@eduardo-VirtualBox: ~/trabalho-grau-a-redes-2$ docker exec -it r1 vtysh
% Can't open configuration file /etc/frr/vtysh.conf due to 'No such file or directory'.

Hello, this is FRRouting (version 8.4.git).
Copyright 1996-2005 Kunihiko Ishiguro, et al.

r1# show ip route
Codes: K - kernel route, C - connected, S - static, R - RIP,
       O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
       T - Table, v - VNC, V - VNC-Direct, A - Babel, F - PBR,
       f - OpenFabric,
       > - selected route, * - FIB route, q - queued, r - rejected, b - backup
       t - trapped, o - offload failure

K>* 0.0.0.0/0 [0/0] via 192.168.10.1, eth1, 00:43:48
C>* 10.0.12.0/29 is directly connected, eth2, 00:43:48
C>* 10.0.14.0/29 is directly connected, eth0, 00:43:48
R>* 10.0.23.0/29 [120/2] via 10.0.12.3, eth2, weight 1, 00:35:27
R>* 10.0.24.0/29 [120/2] via 10.0.12.3, eth2, weight 1, 00:35:27
R>* 10.0.34.0/29 [120/2] via 10.0.14.3, eth0, weight 1, 00:31:41
R>* 10.0.35.0/29 [120/3] via 10.0.12.3, eth2, weight 1, 00:32:46
R>* 10.0.45.0/29 [120/2] via 10.0.14.3, eth0, weight 1, 00:31:41
C>* 192.168.10.0/24 is directly connected, eth1, 00:43:48
R>* 192.168.50.0/24 [120/3] via 10.0.14.3, eth0, weight 1, 00:27:53
r1# show ip route rip
Codes: K - kernel route, C - connected, S - static, R - RIP,
       O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
       T - Table, v - VNC, V - VNC-Direct, A - Babel, F - PBR,
       f - OpenFabric,
       > - selected route, * - FIB route, q - queued, r - rejected, b - backup
       t - trapped, o - offload failure

R>* 10.0.23.0/29 [120/2] via 10.0.12.3, eth2, weight 1, 00:35:35
R>* 10.0.24.0/29 [120/2] via 10.0.12.3, eth2, weight 1, 00:35:35
R>* 10.0.34.0/29 [120/2] via 10.0.14.3, eth0, weight 1, 00:31:49
R>* 10.0.35.0/29 [120/3] via 10.0.12.3, eth2, weight 1, 00:32:54
R>* 10.0.45.0/29 [120/2] via 10.0.14.3, eth0, weight 1, 00:31:49
R>* 192.168.50.0/24 [120/3] via 10.0.14.3, eth0, weight 1, 00:28:01
r1#
```

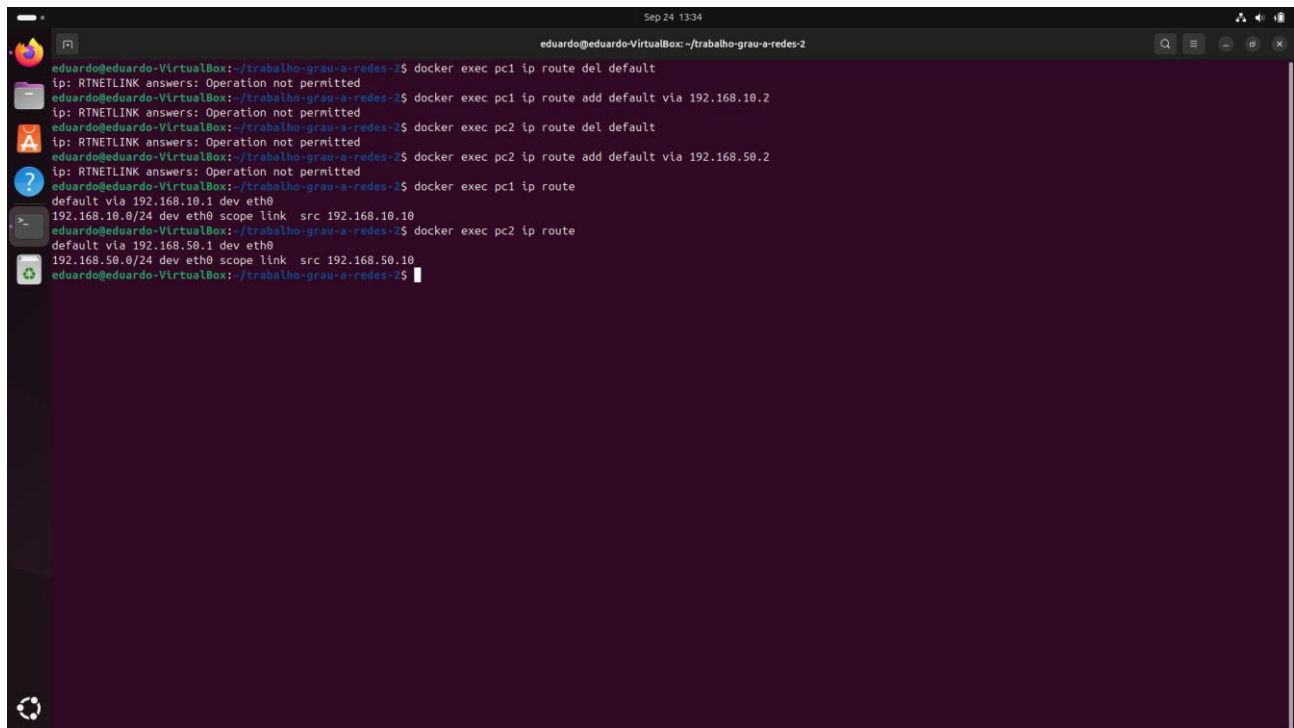
Figura 8 - Informações de roteamento e vizinhança observadas durante a etapa OSPF.

8.1 Alteração do custo OSPF

Para provocar uma mudança de decisão, foi alterado manualmente o custo de uma interface pertencente ao caminho. O valor utilizado foi 100. A lógica do teste foi simples: aumentar o custo de um enlace e consultar novamente a tabela e o traceroute para verificar se o caminho de menor custo acumulado havia mudado.

```
configure terminal
interface <interface do caminho>
ip ospf cost 100
exit
end
```

```
show ip ospf interface
show ip route ospf
traceroute <destino>
```



```
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ docker exec pc1 ip route del default
ip: RTNETLINK answers: Operation not permitted
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ docker exec pc1 ip route add default via 192.168.10.2
ip: RTNETLINK answers: Operation not permitted
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ docker exec pc2 ip route del default
ip: RTNETLINK answers: Operation not permitted
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ docker exec pc2 ip route add default via 192.168.50.2
ip: RTNETLINK answers: Operation not permitted
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ docker exec pc1 ip route
default via 192.168.10.1 dev eth0
192.168.10.0/24 dev eth0 scope link src 192.168.10.10
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ docker exec pc2 ip route
default via 192.168.50.1 dev eth0
192.168.50.0/24 dev eth0 scope link src 192.168.50.10
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$
```

Figura 9 - Etapa de inspeção e alteração de parâmetros de roteamento no FRR.

Esse teste evidencia uma diferença conceitual importante: dois caminhos podem possuir a mesma quantidade de saltos, mas custos OSPF diferentes. Dessa forma, a decisão do OSPF pode mudar sem que o número de roteadores do caminho seja necessariamente o principal fator.

9. Construção do BGP

O BGP foi a etapa mais detalhada do laboratório porque exigiu não apenas conectividade IP, mas também a definição dos Sistemas Autônomos, dos vizinhos BGP, dos anúncios de rede e das políticas necessárias para que as rotas fossem aceitas e propagadas.

9.1 Configuração do R1 - AS100

O R1 pertence ao AS100. Ele estabelece eBGP com roteadores do AS200 e anuncia a rede final 192.168.10.0/24.

```
router bgp 100
neighbor 10.0.12.3 remote-as 200
neighbor 10.0.14.3 remote-as 200
address-family ipv4 unicast
network 192.168.10.0/24
neighbor 10.0.12.3 route-map PERMIT-ALL in
neighbor 10.0.12.3 route-map PERMIT-ALL out
neighbor 10.0.14.3 route-map PERMIT-ALL in
neighbor 10.0.14.3 route-map PERMIT-ALL out
exit-address-family
route-map PERMIT-ALL permit 10
```

9.2 Configuração do R2 - AS200

O R2 faz parte do AS200. Ele possui uma sessão eBGP com o R1 e sessões internas com roteadores do próprio AS200. Durante o diagnóstico, a rede de trânsito 10.0.23.0/29 também precisou ser anunciada para permitir o retorno correto do tráfego.

```
router bgp 200
neighbor 10.0.12.2 remote-as 100
neighbor 10.0.23.3 remote-as 200
neighbor 10.0.24.3 remote-as 200
address-family ipv4 unicast
network 10.0.23.0/29
neighbor 10.0.12.2 route-map PERMIT-ALL in
neighbor 10.0.12.2 route-map PERMIT-ALL out
neighbor 10.0.23.3 next-hop-self
neighbor 10.0.24.3 next-hop-self
exit-address-family
```

9.3 Configuração do R3 - AS200

O R3 foi fundamental no caminho final entre AS300 e AS100. Ele possui ligação interna com o AS200 e eBGP com R5/AS300. A rede 10.0.35.0/29 foi anunciada durante a correção do caminho de retorno.

```
router bgp 200
neighbor 10.0.23.2 remote-as 200
neighbor 10.0.35.3 remote-as 300
address-family ipv4 unicast
network 10.0.35.0/29
neighbor 10.0.23.2 next-hop-self
neighbor 10.0.35.3 next-hop-self
neighbor 10.0.35.3 route-map PERMIT-ALL in
neighbor 10.0.35.3 route-map PERMIT-ALL out
exit-address-family
```

9.4 Configuração do R4 - AS200

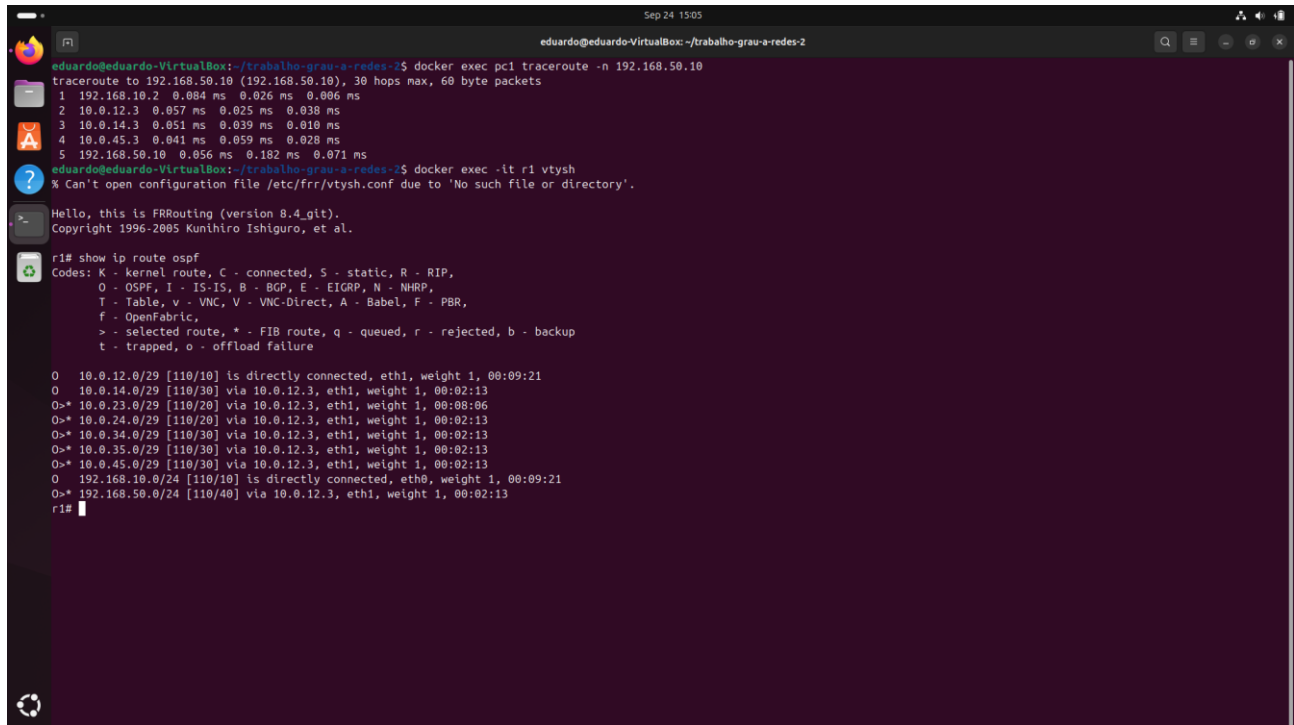
O R4 forneceu um caminho alternativo dentro da topologia. Ele se conecta ao lado do AS100, ao núcleo do AS200 e ao lado do AS300, permitindo redundância de caminho.

```
router bgp 200
neighbor 10.0.14.2 remote-as 100
neighbor 10.0.24.2 remote-as 200
neighbor 10.0.45.3 remote-as 300
address-family ipv4 unicast
neighbor 10.0.24.2 next-hop-self
neighbor 10.0.14.2 route-map PERMIT-ALL in
neighbor 10.0.14.2 route-map PERMIT-ALL out
neighbor 10.0.45.3 route-map PERMIT-ALL in
neighbor 10.0.45.3 route-map PERMIT-ALL out
exit-address-family
```

9.5 Configuração do R5 - AS300

O R5 pertence ao AS300 e representa a outra extremidade do laboratório. Ele anuncia 192.168.50.0/24 e recebe do AS200 a rota para 192.168.10.0/24.

```
router bgp 300
neighbor 10.0.35.2 remote-as 200
neighbor 10.0.45.2 remote-as 200
address-family ipv4 unicast
network 192.168.50.0/24
neighbor 10.0.35.2 route-map PERMIT-ALL in
neighbor 10.0.35.2 route-map PERMIT-ALL out
neighbor 10.0.45.2 route-map PERMIT-ALL in
neighbor 10.0.45.2 route-map PERMIT-ALL out
```



```

eduardo@eduardo-VirtualBox: ~/trabalho-grau-a-redes-2
tracert to 192.168.50.10 (192.168.50.10), 30 hops max, 60 byte packets
 1 192.168.10.2  0.084 ms  0.026 ms  0.006 ms
 2 10.0.12.3  0.057 ms  0.025 ms  0.038 ms
 3 10.0.14.3  0.051 ms  0.039 ms  0.010 ms
 4 10.0.45.3  0.041 ms  0.059 ms  0.020 ms
 5 192.168.50.10  0.056 ms  0.182 ms  0.071 ms
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ docker exec -it r1 vtysh
% Can't open configuration file /etc/frr/vtysh.conf due to 'No such file or directory'.

Hello, this is FRRouting (version 8.4_git).
Copyright 1996-2005 Kunihito Ishiguro, et al.

r1# show ip route ospf
Codes: K - kernel route, C - connected, S - static, R - RIP,
        O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
        T - Table, v - VNC, V - VNC-Direct, A - Babel, F - PBR,
        f - OpenFabric,
        > - selected route, * - FIB route, q - queued, r - rejected, b - backup
        t - trapped, o - offload failure

O  10.0.12.0/29 [110/10] is directly connected, eth1, weight 1, 00:09:21
O  10.0.14.0/29 [110/30] via 10.0.12.3, eth1, weight 1, 00:02:13
O>* 10.0.23.0/29 [110/20] via 10.0.12.3, eth1, weight 1, 00:08:06
O>* 10.0.24.0/29 [110/20] via 10.0.12.3, eth1, weight 1, 00:02:13
O>* 10.0.34.0/29 [110/30] via 10.0.12.3, eth1, weight 1, 00:02:13
O>* 10.0.35.0/29 [110/30] via 10.0.12.3, eth1, weight 1, 00:02:13
O>* 10.0.45.0/29 [110/30] via 10.0.12.3, eth1, weight 1, 00:02:13
O  192.168.10.0/24 [110/10] is directly connected, eth0, weight 1, 00:09:21
O>* 192.168.50.0/24 [110/40] via 10.0.12.3, eth1, weight 1, 00:02:13
r1#

```

Figura 10 - Sessões e informações BGP observadas nos containers.

10. eBGP, iBGP e next-hop-self

As sessões entre AS diferentes são eBGP. No laboratório isso ocorre nas bordas AS100-AS200 e AS200-AS300. Dentro do AS200, as informações precisam circular entre os roteadores internos; por isso foram usadas relações BGP internas.

O comando `next-hop-self` foi aplicado em sessões internas para substituir o próximo salto anunciado por um endereço alcançável do próprio roteador que repassa a rota. Sem esse ajuste, um roteador iBGP pode receber uma rota cujo next-hop original não é alcançável por sua tabela, impedindo que a rota seja usada corretamente.

```
Sep 24 23:34
eduardo@eduardo-VirtualBox: ~/trabalho-grau-a-redes-2

# R2
docker exec r2 vtysh \
-c "configure terminal" \
-c "router ospf" \
-c "network 10.0.12.0/29 area 0" \
-c "network 10.0.23.0/29 area 0" \
-c "network 10.0.24.0/29 area 0"

# R3
docker exec r3 vtysh \
-c "configure terminal" \
-c "router ospf" \
-c "network 10.0.23.0/29 area 0" \
-c "network 10.0.34.0/29 area 0" \
-c "network 10.0.35.0/29 area 0"

# R4
docker exec r4 vtysh \
-c "configure terminal" \
-c "router ospf" \
-c "network 10.0.14.0/29 area 0" \
-c "network 10.0.24.0/29 area 0" \
-c "network 10.0.34.0/29 area 0" \
-c "network 10.0.45.0/29 area 0"

# R5
docker exec r5 vtysh \
-c "configure terminal" \
-c "router ospf" \
-c "network 10.0.35.0/29 area 0" \
-c "network 10.0.45.0/29 area 0" \
-c "network 192.168.50.0/24 area 0"

% Can't open configuration file /etc/frr/vtysh.conf due to 'No such file or directory'.
% Can't open configuration file /etc/frr/vtysh.conf due to 'No such file or directory'.
% Can't open configuration file /etc/frr/vtysh.conf due to 'No such file or directory'.
% Can't open configuration file /etc/frr/vtysh.conf due to 'No such file or directory'.
% Can't open configuration file /etc/frr/vtysh.conf due to 'No such file or directory'.
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ docker exec r1 vtysh -c "show ip ospf neighbor"
% Can't open configuration file /etc/frr/vtysh.conf due to 'No such file or directory'.

Neighbor ID    Pri  State           Up Time    Dead Time Address          Interface          RXmtL  RqstL  DBsmL
10.0.24.2      1  2-Way/DROther   3.163s     36.836s   10.0.12.3         eth1:10.0.12.2     0      0      0
10.0.45.2      1  2-Way/DROther   2.903s     37.096s   10.0.14.3         eth2:10.0.14.2     0      0      0

eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$
```

Figura 11 - Verificação de sessões e rotas BGP após ajustes de propagação.

11. Política eBGP: route-map PERMIT-ALL

Em uma etapa inicial, sessões eBGP apareciam estabelecidas, mas o resumo indicava política e os prefixos não eram propagados como esperado. Para o laboratório foi criada uma route-map permissiva e aplicada nas direções de entrada e saída dos vizinhos eBGP.

```
route-map PERMIT-ALL permit 10
```

```
router bgp <AS>
```

```
address-family ipv4 unicast
```

```
neighbor <vizinho> route-map PERMIT-ALL in
```

```
neighbor <vizinho> route-map PERMIT-ALL out
```

Depois dessa alteração, os resumos BGP passaram a indicar prefixos recebidos nas sessões analisadas. A route-map não foi usada para selecionar uma política complexa; sua função foi permitir explicitamente os anúncios necessários ao experimento.

```
Sep 24 12:18
eduardo@eduardo-VirtualBox: ~/trabalho-grau-e-redes-2

default via 10.0.12.1 dev eth0
10.0.12.0/29 dev eth0 proto kernel scope link src 10.0.12.2
bash-5.1# ping -c 4 10.0.12.3
PING 10.0.12.3 (10.0.12.3): 56 data bytes
64 bytes from 10.0.12.3: seq=0 ttl=64 time=0.291 ms
64 bytes from 10.0.12.3: seq=1 ttl=64 time=0.065 ms
64 bytes from 10.0.12.3: seq=2 ttl=64 time=0.061 ms
64 bytes from 10.0.12.3: seq=3 ttl=64 time=0.058 ms

--- 10.0.12.3 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.058/0.118/0.291 ms
bash-5.1# vtysh
% Can't open configuration file /etc/frr/vtysh.conf due to 'No such file or directory'.

Hello, this is FRRouting (version 8.4_git).
Copyright 1996-2005 Kunihiko Ishiguro, et al.

r1# show ip route
Codes: K - kernel route, C - connected, S - static, R - RIP,
O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
T - Table, V - VNC, V - VNC-Direct, A - Babel, F - PBR,
f - OpenFabric,
> - selected route, * - FIB route, q - queued, r - rejected, b - backup
t - trapped, o - offload failure

K>* 0.0.0.0/0 [0/0] via 10.0.12.1, eth0, 00:09:43
C>* 10.0.12.0/29 is directly connected, eth0, 00:09:43
r1# ping -c 4 10.0.12.3
% Unknown command: ping -c 4 10.0.12.3
r1# ping -c 4 10.0.12.3
% Unknown command: ping -c 4 10.0.12.3
r1# vtysh
% Unknown command: vtysh
r1# show ip route
Codes: K - kernel route, C - connected, S - static, R - RIP,
O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
T - Table, v - VNC, V - VNC-Direct, A - Babel, F - PBR,
f - OpenFabric,
> - selected route, * - FIB route, q - queued, r - rejected, b - backup
t - trapped, o - offload failure

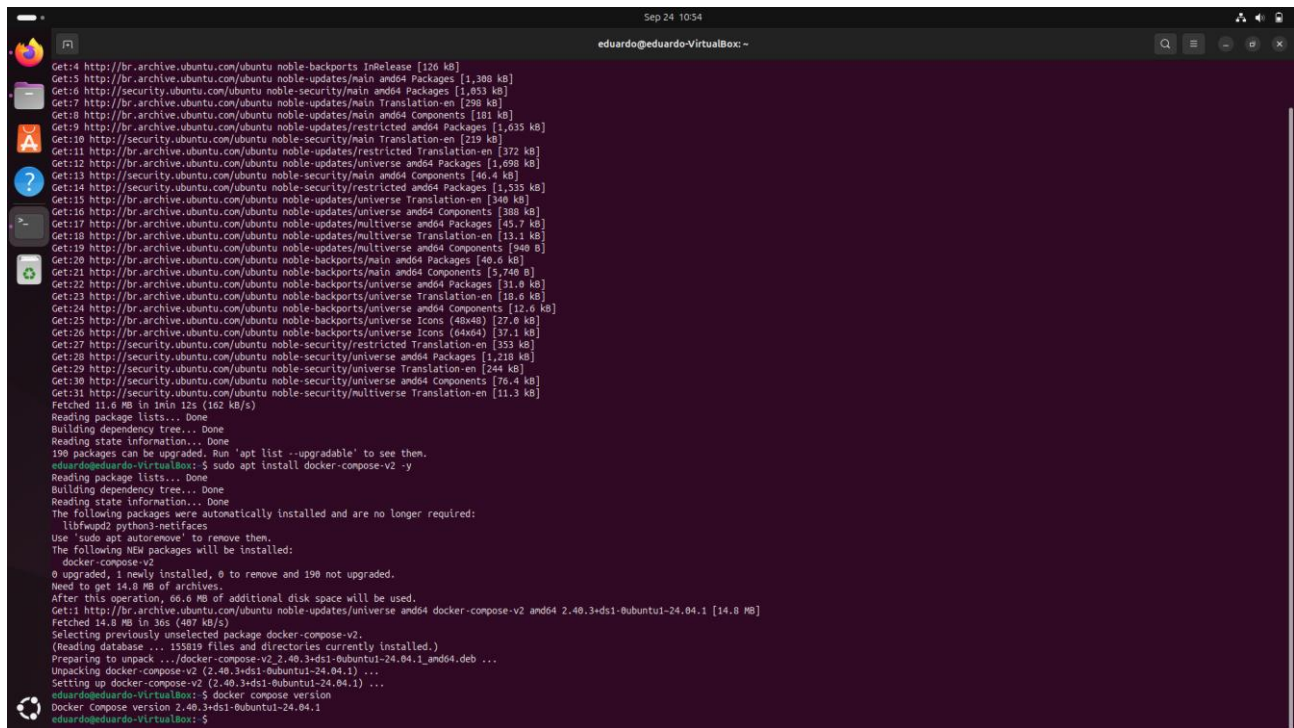
K>* 0.0.0.0/0 [0/0] via 10.0.12.1, eth0, 00:11:52
C>* 10.0.12.0/29 is directly connected, eth0, 00:11:52
r1#
```

Figura 12 - Resumo BGP utilizado para verificar sessões e prefixos recebidos.

12. Como os dados foram coletados

Os resultados não foram inferidos apenas da configuração. Em cada etapa foram usados comandos de observação para confirmar o estado real do plano de controle e do encaminhamento.

Objetivo	Comando/forma de coleta
Ver rotas do kernel/FRR	show ip route
Rotas RIP	show ip route rip
Vizinhos OSPF	show ip ospf neighbor
Rotas OSPF	show ip route ospf
Resumo de sessões BGP	show ip bgp summary
Tabela BGP	show ip bgp
Rotas BGP instaladas	show ip route bgp
Rota específica	show ip route <prefixo>
Conectividade	ping
Caminho IP	traceroute
Execução remota	docker exec <roteador> vtysh -c "<comando>"



```
Get:4 http://br.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://br.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,308 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,453 kB]
Get:7 http://br.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [298 kB]
Get:8 http://br.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [181 kB]
Get:9 http://br.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [1,635 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [219 kB]
Get:11 http://br.archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [372 kB]
Get:12 http://br.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,698 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [46.4 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [1,535 kB]
Get:15 http://br.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [348 kB]
Get:16 http://br.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [388 kB]
Get:17 http://br.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [45.7 kB]
Get:18 http://br.archive.ubuntu.com/ubuntu noble-updates/multiverse Translation-en [13.1 kB]
Get:19 http://br.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:20 http://br.archive.ubuntu.com/ubuntu noble-backports/main amd64 Packages [46.6 kB]
Get:21 http://br.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [5,748 B]
Get:22 http://br.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Packages [11.8 kB]
Get:23 http://br.archive.ubuntu.com/ubuntu noble-backports/universe Translation-en [18.6 kB]
Get:24 http://br.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [12.6 kB]
Get:25 http://br.archive.ubuntu.com/ubuntu noble-backports/universe Icons (48x48) [27.0 kB]
Get:26 http://br.archive.ubuntu.com/ubuntu noble-backports/universe Icons (64x64) [37.1 kB]
Get:27 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [359 kB]
Get:28 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1,218 kB]
Get:29 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [244 kB]
Get:30 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [76.4 kB]
Get:31 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [11.3 kB]
Fetched 11.6 MB in 1min 12s (162 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
190 packages can be upgraded. Run 'apt list --upgradable' to see them.
eduardo@eduardo-VirtualBox:~$ sudo apt install docker-compose-v2 -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libfwpd2 python3-netifaces
Use 'sudo apt autoremove' to remove them.
The following NEW packages will be installed:
  docker-compose-v2
0 upgraded, 1 newly installed, 0 to remove and 190 not upgraded.
Need to get 14.8 MB of archives.
After this operation, 66.6 MB of additional disk space will be used.
Get:1 http://br.archive.ubuntu.com/ubuntu noble-updates/universe amd64 docker-compose-v2 amd64 2.40.3+ds1-0ubuntu1-24.04.1 [14.8 MB]
Fetched 14.8 MB in 36s (407 kB/s)
Selecting previously unselected package docker-compose-v2.
(Reading database ... 155819 files and directories currently installed.)
Preparing to unpack .../docker-compose-v2_2.40.3+ds1-0ubuntu1-24.04.1_and64.deb ...
Unpacking docker-compose-v2 (2.40.3+ds1-0ubuntu1-24.04.1) ...
Setting up docker-compose-v2 (2.40.3+ds1-0ubuntu1-24.04.1) ...
eduardo@eduardo-VirtualBox:~$ docker compose version
Docker Compose version 2.40.3+ds1-0ubuntu1-24.04.1
eduardo@eduardo-VirtualBox:~$
```

Figura 13 - Coleta de dados de roteamento diretamente nos containers Ubuntu/FRR.

13. Rota BGP recebida no AS300

No R5 foi observada a rota para 192.168.10.0/24. O próximo salto era 10.0.35.2 e o AS_PATH observado era 200 100. O código B na tabela de roteamento indica que a rota foi aprendida pelo BGP.

Network: 192.168.10.0/24
Next-hop: 10.0.35.2
AS_PATH: 200 100
Código: B

O AS_PATH deve ser interpretado no contexto do anúncio recebido por R5/AS300: a rota chegou a partir do AS200 e foi originada no AS100. O AS local 300 não aparece no AS_PATH recebido porque ele ainda não foi acrescentado como um AS atravessado no anúncio que R5 está recebendo.

```
eduardo@eduardo-VirtualBox: ~/trabalho-grau-a-redes-2$ docker exec -it r2 vtysh
% Can't open configuration file /etc/frr/vtysh.conf due to 'No such file or directory'.

Hello, this is FRRouting (version 8.4_git).
Copyright 1996-2005 Kunihito Ishiguro, et al.

r2# configure terminal
r2(config)#
r2(config)# router bgp 200
r2(config-router)# bgp router-id 2.2.2.2
r2(config-router)#
r2(config-router)# neighbor 10.0.12.2 remote-as 100
r2(config-router)#
r2(config-router)# end
r2# write memory
Note: this version of vtysh never writes vtysh.conf
Building Configuration...
Configuration saved to /etc/frr/zebra.conf
Configuration saved to /etc/frr/ospfd.conf
Configuration saved to /etc/frr/bgpd.conf
Configuration saved to /etc/frr/statd.conf
r2# show ip bgp summary

IPv4 Unicast Summary (VRF default):
BGP router identifier 2.2.2.2, local AS number 200 vrf-id 0
BGP table version 0
RIB entries 0, using 0 bytes of memory
Peers 1, using 717 KiB of memory

Neighbor      V    AS  MsgRcvd  MsgSent  TblVer  InQ  OutQ  Up/Down  State/PfxRcd  PfxSnt Desc
10.0.12.2      4    100        3         3        0     0     0 00:00:23      (Policy) (Policy) N/A

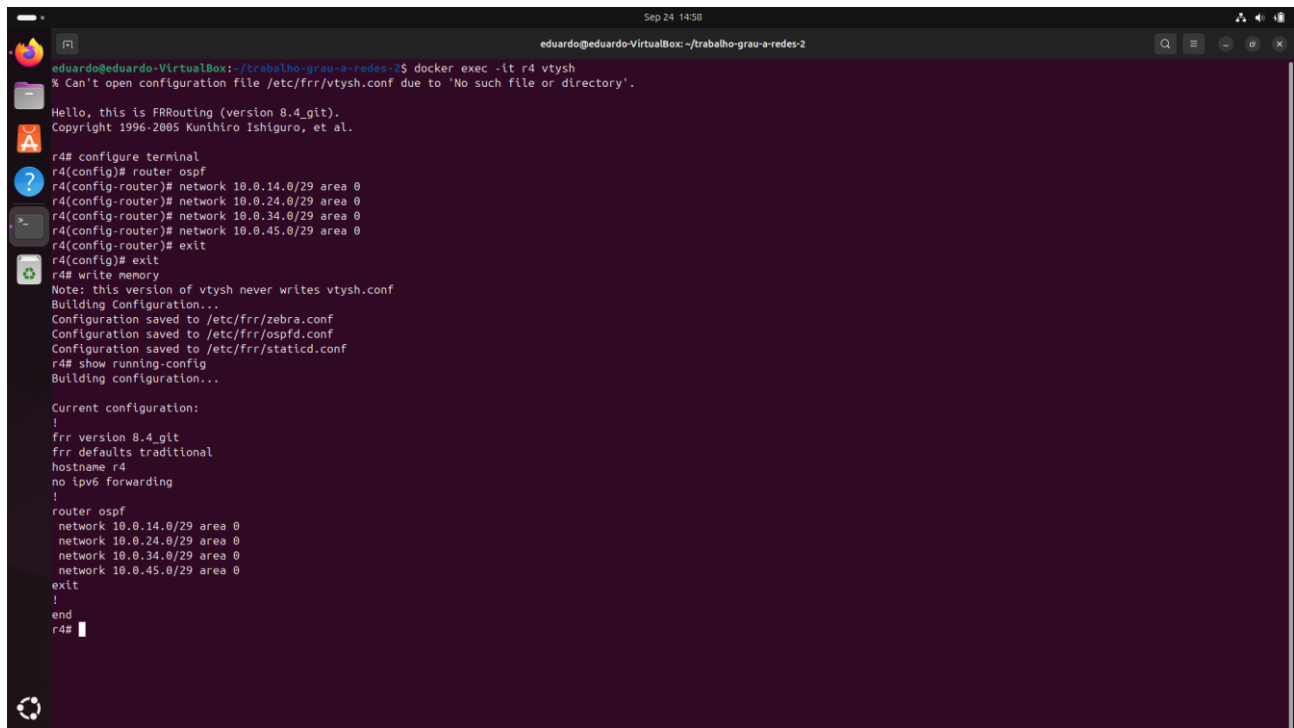
Total number of neighbors 1
r2#
```

Figura 14 - Tabela/estado BGP utilizado na análise da rota recebida.

14. Falha inicial: rota de ida existia, mas o ping falhava

Um dos pontos mais importantes do trabalho foi a depuração de uma falha real. Em determinado momento, o R5 possuía a rota BGP para 192.168.10.0/24, mas o ping para 192.168.10.2 apresentava 100% de perda. Isso mostrou que visualizar uma rota na origem não garante conectividade fim a fim.

A hipótese investigada foi a ausência de caminho de retorno. O pacote saía do AS300 e avançava até o AS200, mas a resposta precisava retornar ao endereço de origem atravessando as redes de trânsito. Por isso, as tabelas foram verificadas roteador por roteador.



```
eduardo@eduardo-VirtualBox: ~/trabalho-grau-a-redes-2$ docker exec -it r4 vtysh
% Can't open configuration file /etc/frr/vtysh.conf due to 'No such file or directory'.
Hello, this is FRRouting (version 8.4_git).
Copyright 1996-2005 Kunihito Ishiguro, et al.

r4# configure terminal
r4(config)# router ospf
r4(config-router)# network 10.0.14.0/29 area 0
r4(config-router)# network 10.0.24.0/29 area 0
r4(config-router)# network 10.0.34.0/29 area 0
r4(config-router)# network 10.0.45.0/29 area 0
r4(config-router)# exit
r4(config)# exit
r4# write memory
Note: this version of vtysh never writes vtysh.conf
Building Configuration...
Configuration saved to /etc/frr/zebra.conf
Configuration saved to /etc/frr/ospfd.conf
Configuration saved to /etc/frr/staticd.conf
r4# show running-config
Building configuration...

Current configuration:
!
frr version 8.4_git
frr defaults traditional
hostname r4
no ipv6 forwarding
!
router ospf
 network 10.0.14.0/29 area 0
 network 10.0.24.0/29 area 0
 network 10.0.34.0/29 area 0
 network 10.0.45.0/29 area 0
exit
!
end
r4#
```

Figura 15 - Estado intermediário utilizado durante o diagnóstico da falha de conectividade.

15. Diagnóstico roteador por roteador

Primeiro foi verificado o encaminhamento IPv4 nos roteadores intermediários. O valor de `ip_forward` estava habilitado nos pontos analisados, descartando a hipótese de o kernel Linux estar bloqueando o encaminhamento.

Depois, a rota para 192.168.10.0/24 foi acompanhada dentro do AS200. O R2 conseguia alcançar 192.168.10.2, enquanto o R3 inicialmente não conseguia completar o teste. Essa diferença direcionou a investigação para as redes intermediárias e para o retorno.

Foi constatado que o R1 não possuía conhecimento suficiente para retornar tráfego para a rede 10.0.23.0/29. A rede foi então anunciada no BGP pelo R2.

```
router bgp 200
 address-family ipv4 unicast
  network 10.0.23.0/29
 exit-address-family
```

Após o anúncio, o R1 passou a conhecer 10.0.23.0/29 via BGP, e o teste do R3 para 192.168.10.2 passou a funcionar.

```
eduardo@eduardo-VirtualBox: ~/trabalho-grau-a-redes-2$ docker exec -it r2 vtysh
% Can't open configuration file /etc/frr/vtysh.conf due to 'No such file or directory'.

Hello, this is FRRouting (version 8.4_git).
Copyright 1996-2005 Kunihito Ishiguro, et al.

r2# configure terminal
r2(config)#
r2(config)# router bgp 200
r2(config-router)#
r2(config-router)# neighbor 10.0.23.3 remote-as 200
r2(config-router)# neighbor 10.0.24.3 remote-as 200
r2(config-router)#
r2(config-router)# end
r2# write memory
Note: this version of vtysh never writes vtysh.conf
Building Configuration...
Configuration saved to /etc/frr/zebra.conf
Configuration saved to /etc/frr/ospfd.conf
Configuration saved to /etc/frr/bgpd.conf
Configuration saved to /etc/frr/statd.conf
r2# show ip bgp summary

IPv4 Unicast Summary (VRF default):
BGP router identifier 2.2.2.2, local AS number 200 vrf-id 0
BGP table version 0
RIB entries 0, using 0 bytes of memory
Peers 3, using 2151 KiB of memory

Neighbor      V      AS  MsgRcvd  MsgSent  TblVer  InQ  OutQ  Up/Down  State/PfxRcd  PfxSnt Desc
10.0.12.2      4      100       16       16        0    0    0 00:13:28 (Policy) (Policy) N/A
10.0.23.3      4      200        0        1        0    0    0 never Active      0 N/A
10.0.24.3      4      200        0        1        0    0    0 never Active      0 N/A

Total number of neighbors 3
r2#
```

Figura 16 - Consulta de rotas durante a correção do caminho intermediário.

16. Segunda correção: rede 10.0.35.0/29

Mesmo após a primeira correção, o teste iniciado no R5 ainda precisava de um retorno para a rede de trânsito entre R3 e R5. O R1 não conhecia 10.0.35.0/29. Por isso, essa rede também foi anunciada no BGP pelo R3.

```
router bgp 200
address-family ipv4 unicast
network 10.0.35.0/29
exit-address-family
```

Depois desse anúncio, o R1 passou a possuir uma rota BGP para 10.0.35.0/29. Esse foi o ajuste que completou o caminho de retorno necessário ao ping originado no R5.

```
Sep 28 12:20
eduardo@eduardo-VirtualBox: ~/trabalho-grau-e-redes-2

64 bytes from 192.168.50.2: seq=12 ttl=63 time=0.069 ms
64 bytes from 192.168.50.2: seq=13 ttl=63 time=0.071 ms
64 bytes from 192.168.50.2: seq=14 ttl=63 time=0.069 ms
64 bytes from 192.168.50.2: seq=15 ttl=63 time=0.067 ms
64 bytes from 192.168.50.2: seq=16 ttl=63 time=0.069 ms
64 bytes from 192.168.50.2: seq=17 ttl=63 time=0.069 ms
64 bytes from 192.168.50.2: seq=18 ttl=63 time=0.079 ms
64 bytes from 192.168.50.2: seq=19 ttl=63 time=0.061 ms

--- 192.168.50.2 ping statistics ---
20 packets transmitted, 20 packets received, 0% packet loss
round-trip min/avg/max = 0.061/0.075/0.188 ms

=====
RIP - R5 -> 192.168.10.2
=====
traceroute to 192.168.10.2 (192.168.10.2), 30 hops max, 46 byte packets
 1  r4.trabalho-grau-a-redes-2_rede_r4_r5 (10.0.45.2)  0.003 ms  0.003 ms  0.002 ms
 2  192.168.10.2 (192.168.10.2)  0.001 ms  0.003 ms  0.004 ms
PING 192.168.10.2 (192.168.10.2): 56 data bytes
64 bytes from 192.168.10.2: seq=0 ttl=63 time=0.715 ms
64 bytes from 192.168.10.2: seq=1 ttl=63 time=0.063 ms
64 bytes from 192.168.10.2: seq=2 ttl=63 time=0.064 ms
64 bytes from 192.168.10.2: seq=3 ttl=63 time=0.070 ms
64 bytes from 192.168.10.2: seq=4 ttl=63 time=0.070 ms
64 bytes from 192.168.10.2: seq=5 ttl=63 time=0.068 ms
64 bytes from 192.168.10.2: seq=6 ttl=63 time=0.069 ms
64 bytes from 192.168.10.2: seq=7 ttl=63 time=0.198 ms
64 bytes from 192.168.10.2: seq=8 ttl=63 time=0.068 ms
64 bytes from 192.168.10.2: seq=9 ttl=63 time=0.068 ms
64 bytes from 192.168.10.2: seq=10 ttl=63 time=0.068 ms
64 bytes from 192.168.10.2: seq=11 ttl=63 time=0.080 ms
64 bytes from 192.168.10.2: seq=12 ttl=63 time=0.071 ms
64 bytes from 192.168.10.2: seq=13 ttl=63 time=0.068 ms
64 bytes from 192.168.10.2: seq=14 ttl=63 time=0.068 ms
64 bytes from 192.168.10.2: seq=15 ttl=63 time=0.071 ms
64 bytes from 192.168.10.2: seq=16 ttl=63 time=0.071 ms
64 bytes from 192.168.10.2: seq=17 ttl=63 time=0.069 ms
64 bytes from 192.168.10.2: seq=18 ttl=63 time=0.069 ms
64 bytes from 192.168.10.2: seq=19 ttl=63 time=0.070 ms

--- 192.168.10.2 ping statistics ---
20 packets transmitted, 20 packets received, 0% packet loss
round-trip min/avg/max = 0.063/0.107/0.715 ms
eduardo@eduardo-VirtualBox: ~/trabalho-grau-a-redes-2$
```

Figura 17 - Verificação das rotas após os anúncios adicionais das redes de trânsito.

17. Validação final de conectividade

Com as rotas de ida e retorno disponíveis, o teste foi repetido a partir do R5/AS300 para 192.168.10.2, na rede do AS100. O resultado final comprovou conectividade fim a fim.

4 packets transmitted
4 packets received
0% packet loss

RTT aproximado:
mínimo = 0,070 ms
médio = 0,093 ms
máximo = 0,157 ms

O RTT médio de aproximadamente 0,093 ms é coerente com um laboratório inteiramente virtualizado na mesma VM, pois não existe a latência física de enlaces geograficamente separados. Esse número é um resultado do ambiente experimental e não deve ser generalizado para redes BGP reais da Internet.

```
eduardo@eduardo-VirtualBox: ~/trabalho-grau-a-redes-2$ docker restart r1 r2 r3 r4 r5
r1
r2
r3
r4
r5
eduardo@eduardo-VirtualBox: ~/trabalho-grau-a-redes-2$ docker exec r1 vtysh -c "show ip rip"
% Can't open configuration file /etc/frr/vtysh.conf due to 'No such file or directory'.
Codes: R - RIP, C - connected, S - Static, O - OSPF, B - BGP
Sub-codes:
(n) - normal, (s) - static, (d) - default, (r) - redistribute,
(i) - interface

Network        Next Hop        Metric From      Tag Time
C(i) 10.0.12.0/29 0.0.0.0         1 self           0
C(i) 10.0.14.0/29 0.0.0.0         1 self           0
R(n) 10.0.23.0/29 10.0.12.3       2 10.0.12.3      0 02:58
R(n) 10.0.24.0/29 10.0.12.3       2 10.0.12.3      0 02:58
R(n) 10.0.34.0/29 10.0.14.3       2 10.0.14.3      0 02:54
R(n) 10.0.35.0/29 10.0.14.3       3 10.0.14.3      0 02:54
R(n) 10.0.45.0/29 10.0.14.3       2 10.0.14.3      0 02:54
C(i) 192.168.10.0/24 0.0.0.0         1 self           0
R(n) 192.168.50.0/24 10.0.14.3       3 10.0.14.3      0 02:54
eduardo@eduardo-VirtualBox: ~/trabalho-grau-a-redes-2$
```

Figura 18 - Resultados de validação coletados no terminal Ubuntu.

18. Traceroute final e caminho efetivamente utilizado

O traceroute final permitiu observar os saltos IP realmente usados no encaminhamento. O caminho final foi:

```
R5 / AS300
↓
R3 - 10.0.35.2 / AS200
↓
R2 - 10.0.23.2 / AS200
↓
192.168.10.2 / rede do AS100
```

Esse resultado substitui caminhos intermediários observados durante a configuração. O caminho acima corresponde à validação final depois das correções de roteamento.

```
eduardo@eduardo-VirtualBox: ~/trabalho-grau-a-redes-2
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ for r in r2 r3 r4 r5; do
echo "===== $r ====="
docker exec $r ip -4 -br addr
done
===== r2 =====
lo                UNKNOWN    127.0.0.1/8
eth0@if129        UP          10.0.12.3/29
eth1@if130        UP          10.0.23.2/29
eth2@if131        UP          10.0.24.2/29
===== r3 =====
lo                UNKNOWN    127.0.0.1/8
eth0@if132        UP          10.0.35.2/29
eth1@if136        UP          10.0.23.3/29
eth2@if139        UP          10.0.34.2/29
===== r4 =====
lo                UNKNOWN    127.0.0.1/8
eth0@if133        UP          10.0.34.3/29
eth1@if137        UP          10.0.45.2/29
eth2@if142        UP          10.0.14.3/29
eth3@if144        UP          10.0.24.3/29
===== r5 =====
lo                UNKNOWN    127.0.0.1/8
eth0@if135        UP          10.0.35.3/29
eth1@if141        UP          10.0.45.3/29
eth2@if143        UP          192.168.50.2/24
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$
```

Figura 19 - Saídas finais de roteamento utilizadas para confirmar o caminho no laboratório.

19. Diferença entre AS_PATH e traceroute

AS_PATH e traceroute descrevem o caminho em níveis diferentes. O AS_PATH 200 100 é um atributo do BGP e registra Sistemas Autônomos presentes no anúncio da rota. Já o traceroute mostra saltos IP concretos percorridos pelos pacotes.

Informação	Resultado final	Interpretação
AS_PATH	200 100	O anúncio recebido no AS300 passou pelo AS200 e teve origem no AS100
Traceroute	10.0.35.2 → 10.0.23.2 → 192.168.10.2	Mostra R3 e R2 individualmente dentro do AS200

Por isso, o fato de o traceroute mostrar dois roteadores no AS200 não faz o número 200 aparecer duas vezes no AS_PATH. O atributo BGP representa a sequência de AS, não a lista de todos os roteadores.

20. Comparação técnica entre RIP, OSPF e BGP

Característica	RIP	OSPF	BGP
Família	Vetor de distância	Estado de enlace	Vetor de caminho
Decisão principal	Número de saltos	Custo acumulado	Atributos e política BGP
Código de rota	R	O	B
Escopo típico	Roteamento interno	Roteamento interno	Entre Sistemas Autônomos
Elemento observado	Métrica por saltos	Custo e adjacências	AS_PATH, next-hop e sessões
Mudança provocada	Propagação das redes	Custo de interface = 100	Anúncios/políticas e rotas de retorno

No laboratório, a diferença mais visível entre RIP e OSPF foi a forma de escolha: RIP relaciona a métrica à quantidade de saltos, enquanto OSPF considera custo. No BGP, a análise mudou de nível porque passou a envolver AS, sessões e atributos como AS_PATH.

21. Métricas efetivamente comprovadas no Ubuntu

Para manter o relatório fiel às evidências, a tabela abaixo não preenche com números aquilo que não foi medido de forma padronizada nos prints.

Métrica	RIP	OSPF	BGP
Rotas	Tabelas/rotas observadas no FRR	Tabelas/rotas observadas no FRR	192.168.10.0/24 aprendida no R5
Pacotes de controle	Não quantificado	Não quantificado	Não quantificado
Bytes de controle	Não quantificado	Não quantificado	Não quantificado
RTT médio	Sem valor comparável registrado	Sem valor comparável registrado	0,093 ms no teste final
Convergência	Não cronometrada	Não cronometrada	Não cronometrada
Caminho final	Observado durante teste	Reavaliado após custo	R5 → R3 → R2 → 192.168.10.2
Perda de pacotes	Sem série comparável registrada	Sem série comparável registrada	0% no teste final

Essa limitação é relevante: para comparar quantitativamente consumo de controle e convergência seria necessário executar uma coleta planejada para os três protocolos, com a mesma duração, mesma topologia e mesma metodologia. Como essa coleta não aparece nas evidências disponíveis, o relatório não apresenta valores fictícios.

22. Erros e problemas encontrados

Problema	Efeito observado	Como foi tratado
Aviso /etc/frr/vtysh.conf ausente	Mensagem aparecia ao usar vtysh	Comandos continuaram funcionando; aviso não impediu o experimento
Política eBGP	Sessões podiam estar ativas sem troca esperada de prefixos	route-map PERMIT-ALL aplicada em entrada/saída
Next-hop em iBGP	Rotas propagadas internamente podiam manter next-hop inadequado	Uso de next-hop-self
R5 conhecia 192.168.10.0/24 mas ping falhava	100% de perda	Investigação do caminho de retorno
R1 sem retorno para 10.0.23.0/29	R3 não completava comunicação	Anúncio 10.0.23.0/29 pelo R2
R1 sem retorno para 10.0.35.0/29	R5 ainda não completava ping	Anúncio 10.0.35.0/29 pelo R3

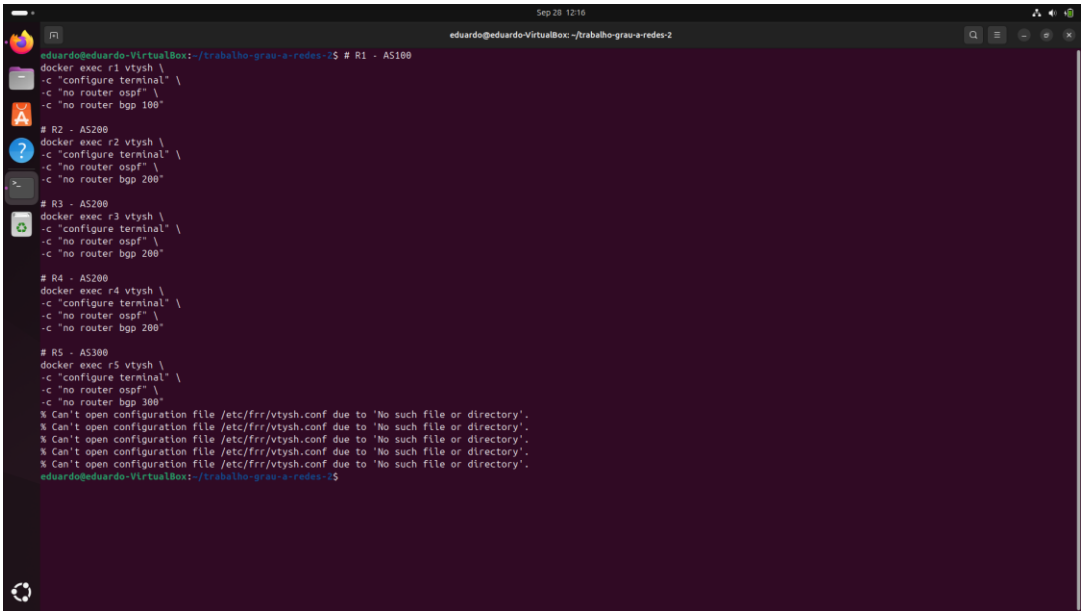


Figura 20 - Terminal Ubuntu durante uma das etapas de diagnóstico e correção.

23. Sequência recomendada para reproduzir o laboratório

1. Inicializar a VM Ubuntu e confirmar conectividade/ambiente Docker.
2. Subir os cinco containers que representam R1, R2, R3, R4 e R5.
3. Confirmar interfaces e endereços IP de cada container.
4. Habilitar encaminhamento IPv4 quando necessário.
5. Confirmar que o FRRouting está em execução e acessar o vtysh.
6. Aplicar um protocolo por vez, evitando misturar resultados de RIP, OSPF e BGP.
7. No RIP, anunciar redes diretamente conectadas e consultar show ip route rip.
8. No OSPF, formar adjacências, consultar custos e alterar uma interface para custo 100.
9. No BGP, definir AS100, AS200 e AS300 e configurar vizinhos eBGP/iBGP.
10. Aplicar next-hop-self nas sessões internas necessárias.
11. Aplicar a route-map permissiva nas sessões eBGP do laboratório.
12. Verificar show ip bgp summary e show ip bgp antes de testar o plano de dados.
13. Confirmar a rota 192.168.10.0/24 no R5 e seu AS_PATH 200 100.
14. Executar ping e traceroute.
15. Se houver falha, inspecionar também o caminho de retorno, roteador por roteador.
16. Anunciar as redes de trânsito necessárias e repetir a validação.
17. Registrar o resultado final somente depois de obter conectividade fim a fim.

24. Comandos finais de verificação

```
# Containers
docker ps

# Sessões BGP em todos os roteadores
for r in r1 r2 r3 r4 r5; do
    echo "===== $r ====="
    docker exec $r vtysh -c "show ip bgp summary"
done

# R5 / AS300
docker exec r5 vtysh -c "show ip bgp"
docker exec r5 vtysh -c "show ip route bgp"
docker exec r5 vtysh -c "show ip route 192.168.10.0/24"

# Plano de dados
docker exec r5 ping -c 4 192.168.10.2
docker exec r5 traceroute 192.168.10.2
```

Essa sequência permite demonstrar, em poucos comandos, a existência dos cinco roteadores, o estado das sessões, a rota recebida pelo AS300 e o caminho real até o AS100.

25. Resultado final consolidado

Item	Resultado comprovado
Plataforma	Ubuntu Linux + Docker + FRRouting
Roteadores	5 containers: R1, R2, R3, R4 e R5

Sistemas Autônomos	AS100, AS200 e AS300
Rede aprendida no R5	192.168.10.0/24
Próximo salto	10.0.35.2
AS_PATH	200 100
Código na tabela	B
Ping final	4 enviados, 4 recebidos, 0% de perda
RTT médio final	aprox. 0,093 ms
Traceroute final	R5 → R3 → R2 → 192.168.10.2

O principal resultado técnico não foi apenas conseguir um ping. O processo mostrou como o plano de controle e o plano de dados precisam ser analisados em conjunto. Uma rota BGP podia existir no R5 e ainda assim a comunicação falhar por falta de retorno. A correção exigiu acompanhar o pacote através das redes intermediárias e tornar as redes de trânsito conhecidas no sentido inverso.

26. Conclusão

O laboratório demonstrou que uma VM Ubuntu pode ser utilizada como plataforma completa para estudo de roteamento. O Docker forneceu isolamento suficiente para representar cinco roteadores independentes, enquanto o FRRouting forneceu os protocolos, as sessões e as tabelas necessárias para reproduzir comportamentos encontrados em redes reais.

RIP, OSPF e BGP foram analisados por perspectivas diferentes: saltos no RIP, custo e adjacências no OSPF, e Sistemas Autônomos, atributos e políticas no BGP. A etapa BGP foi particularmente útil porque expôs problemas que não aparecem quando se observa apenas a rota de ida.

A correção das redes 10.0.23.0/29 e 10.0.35.0/29 mostrou, na prática, a importância do caminho de retorno. Depois dos ajustes, o R5/AS300 alcançou 192.168.10.2 no AS100 com 0% de perda, e o traceroute confirmou o caminho R5 → R3 → R2 → destino.

Por fim, o experimento reforça que a validação de roteamento deve combinar configuração, estado dos protocolos, tabela de roteamento, ping e traceroute. Nenhuma dessas evidências isoladamente descreve todo o funcionamento da rede.